

Is the payload ready to ship?

Talon Robotics — Payload Deployment Delivery

Analyst case study · Project 4 of 5

Prepared for: the programme board · Reporting against build 88 (RC1), 30 September 2026 Stack: T-SQL (SQL Server) · Power BI (DAX) · Excel

Data disclosure. Talon Robotics is a fictional mechatronics company. All data in this case study is synthetic and generated by the scripts in `sql/`. No confidential, proprietary or production data is used, and no claim is made about any real programme.

01 · Executive summary

The programme reports **93.86% complete**. It is **51.96% ready to ship**.

Both figures are correct. They answer different questions, and only one of them is the question on the agenda.

Work items closed	93.86%
Requirements with passing evidence	96.53%
Must-ship requirements ready to ship	51.96%
The gap	41.90 points

♦ **Why the gap exists.** A verification is evidence about **one build**. If a requirement passed on build 62 and its subsystem changed in build 71, the green tick in the requirements tool is describing software that no longer exists. Of the 519 baselined requirements, 96.53% have a passing test somewhere in their history — and a quarter of the evidence that meets policy has been overtaken by change since it was captured.

The gap is not spread evenly. Two safety-critical subsystems — Flight Control Interface and Payload Release Mechanism — carry **109 of the 281 must-ship requirements**, changed in 45 and 42 builds respectively, and sit at **23.40%** and **32.26%** ready. Everything else is between 59% and 83%.

The honest schedule answer: 226 requirements outstanding, **425 hardware-rig hours — 2.4 rig-weeks** at current capacity — plus **272 airframe hours** (34 flights) that no rig can do. Sixty-nine of them can be booked in the first rig week.

02 · The business question, and why the obvious metric cannot answer it

Is the new payload ready to ship, and what is still open?

The programme already had a number for this: work-item completion, 93.86%. It is a real metric, honestly produced, and it is not an answer to the question — because it counts **effort**, and shipping depends on **evidence**.

Three things have to be true of a requirement before it can ship:

- **Evidence exists.** Somebody ran a test and it passed.
- **The evidence counts.** It was produced at the grade the requirement's type demands, by somebody entitled to produce it.
- **The evidence still applies.** Nothing it depended on has moved since.

Work-item completion tests none of these. It tests whether the task was ticked.

Scope

540 requirements (519 baselined, 21 withdrawn) across 8 subsystems, 1,354 test cases, 3,011 test runs over 88 builds spanning January 2025 to September 2026, 1,091 work items and a 96-item RAID register. Out of scope: cost and effort forecasting, requirement decomposition, and failure taxonomy — all recorded in the traceability matrix as deliberate boundaries rather than omissions.

03 · Methodology, and the one modelling decision everything rests on

A verification is a dated fact against a build, not an attribute of a requirement.

The obvious model puts a `Verified` flag, or a `VerifiedDate`, on the requirement row. It is simpler, it is what most requirements tools actually do, and it makes the central question of this analysis impossible to ask.

So `Fact_TestRun` carries both a test case and a **build**, and `Fact_BuildSubsystemChange` records which subsystems each build touched. Staleness becomes **computable** rather than asserted:

A verification is **current** when nothing it depended on has changed since it ran. It is **stale** if a later build changed the same subsystem, or the requirement itself was *modified* after the run.

✦ **What is deliberately not a staleness rule: age.** A verification from fourteen months ago is perfectly good if nothing underneath it has moved, and one from last Tuesday is worthless if the subsystem was rewritten on Wednesday. Elapsed time is the proxy people reach for because it is easy to compute. Intervening change is the actual question, and it is why the model needs build history rather than timestamps.

And *clarified* is not *modified*. Rewording a requirement for a certification reviewer does not change the behaviour under test. Treating it as invalidated evidence would inflate the finding and cost it credibility on the part that is real. The model records the distinction and only one of them invalidates.

Validation

Every headline figure is implemented twice — once in T-SQL, once in Excel formulas — and reconciled on the workbook's own Validation sheet: **17 of 17 agree**. The Power BI model is queried in DAX and reconciled against SQL on **30 figures**, including the state partition, and readiness and the stale split for all eight subsystems individually. The acceptance suite is **25 of 25**, and the generator is deterministic, so every figure here is reproducible by a reader who runs the scripts.

Agreement between tools is necessary, not sufficient — two implementations derived from the same flawed definition will agree with each other and both be wrong. That caveat is stated in the validation report, the workbook and the Power BI validator, because it is the one that is easiest to forget while looking at a column of green.

04 · Finding 1 — Most of the evidence is real. A quarter of it no longer applies.

Verification state	Requirements	Of which must-ship
Current	293	146
Stale — subsystem changed	88	54
Stale — requirement changed	7	3

Verification state	Requirements	Of which must-ship
Insufficient evidence	113	70
No evidence at all	18	8
Total baselined	519	281

Only **18 requirements** — 3.5% — have never been tested. That is what makes the headline coverage figure look healthy, and it is genuinely good news.

The problem is the 208 requirements that *have* been tested and still cannot be shipped on:

- **95 have evidence that has been overtaken** by a change to the same subsystem, or by a change to the requirement itself. 24.48% of all policy-compliant evidence.
- **113 have evidence that never counted** — produced below the grade the requirement's type demands, or by the requirement's own author.

A requirement that has been tested and a requirement that is verified are not the same population, and the difference is 45% of the must-ship set (127 of 281).

05 · Finding 2 — The gap sits exactly where the design is still moving

Subsystem	Criticality	Must-ship	Current	Readiness	Builds changed
Flight Control Interface	Safety	47	11	23.40%	45
Payload Release Mechanism	Safety	62	20	32.26%	42
Power Management	Mission	37	22	59.46%	15
Command & Telemetry Link	Mission	43	26	60.47%	23
Structural Interface	Safety	19	13	68.42%	15
Ground Control Software	Support	28	20	71.43%	21
Navigation & Sensor Fusion	Mission	33	24	72.73%	17
Diagnostics & Logging	Support	12	10	83.33%	18

The two worst subsystems are also the two most-changed, and they carry **39% of everything that must ship**.

✦ **This is not a testing problem.** A subsystem verified once and then changed in forty-five subsequent builds is not behind on test execution — it is **still being designed**. Re-running its test suite today buys evidence that the next build invalidates. That is a different conversation, with a different owner, and a different remedy: stop changing it, or accept that its verification closes only after it stops.

The programme has been treating both as "testing is behind".

06 · Finding 3 — 113 requirements are verified by evidence that does not count

`Ref_VerificationPolicy` states, per requirement type, what counts as evidence. It is the most contestable input in the analysis, so it lives in a table a systems engineer can argue with rather than inside a query nobody reads.

Requirement type	Minimum evidence	Independent tester
Safety	Hardware-in-the-loop	required
Regulatory	Field (on the airframe)	required
Performance	Hardware-in-the-loop	—
Functional	Integration	—
Interface	Integration	—

Against that policy:

- **110 requirements** have passing evidence below the level their type demands, and none that meets it — a safety behaviour signed off by a unit test.
- **20 requirements** (19 of them must-ship) have a pass by their own owner and no admissible evidence, against a policy requiring an independent witness. The two groups overlap by 17, so together they are the 113. (46 requirements carry at least one self-run pass; the other 26 also hold evidence that counts.)

Both look like a pass on every dashboard the programme currently runs. Neither is admissible evidence, and for the Regulatory subset neither would be accepted by the certifying authority.

Policy compliance: 77.45% of requirements with evidence have evidence that actually counts.

07 · Finding 4 — Rig contention is not a separate problem. It is the cause.

Approaching a release candidate the programme re-ran its regression suite. Anything re-run on the release candidate is current by definition — that is what the exercise is for.

What did not get re-run is whatever needed the scarce resource:

Test level	Made the RC regression	Why
Unit	~75%	Automated, effectively free
Integration	~74%	Automated, cheap
Hardware-in-the-loop	~56%	Six rigs, bookable hours
Field (airframe)	~37%	One airframe, weather, crew

So stale evidence lands hardest on exactly the Safety and Regulatory requirements whose policy **demands** hardware evidence in the first place.

♦ **This reframes the whole problem.** "Verification is behind" and "we cannot get rig time" are not two items on a risk register — they are one item, and the second causes the first. The recommended action for most of the outstanding queue is therefore *book rig time*, not *write more tests*. The programme has been resourcing the wrong side of it.

08 · The verification queue as an operational control

226 outstanding requirements is not a plan. A programme can book perhaps 180 rig hours in a week — six rigs at thirty bookable hours — so the queue stays complete for audit and marks what is actually schedulable.

69 requirements in the first rig week. 425 rig hours, or 2.4 rig-weeks, to clear the rig backlog — and 272 airframe hours, 34 flights, that no rig can do.

Each item is costed by what its action needs, not by the tests it happens to have: every test at or above the level its policy demands, one new test where none exists yet at that level (91 items need one written first; pricing each at a single run is a floor), and nothing for a signature. Field evidence needs the airframe — one aircraft, weather and a crew — so its hours are reported beside the rig figure and never divided by rig capacity, and no item that needs a flight is booked into the rig

week. The data holds no airframe capacity, so those 272 hours are costed, not scheduled; with one aircraft they are the likelier constraint.

Action	What it means	Who does it
RERUN	Evidence was valid; the subsystem has moved since	Test engineer, needs a rig slot
RAISE_TEST_LEVEL	Passing evidence sits below the required grade	Test engineer: a rig slot or a flight, often a new test first
REVIEW_THEN_RERUN	The requirement itself was modified after verification	Systems engineer first , then test
INDEPENDENT_WITNESS	Verified by its own author	A second engineer's signature; a witnessed re-run if the evidence has since been overtaken
VERIFY	No passing evidence exists at all	Write the test, then run it

The action follows the state, not the rank. Sending a team to re-run a test when the *requirement* changed risks certifying against the old wording — so that case goes to a systems engineer before it goes near a rig. And `INDEPENDENT_WITNESS` costs a signature, not rig time, when nothing has changed since the pass — two of the three here. The third passed on build 57, and its subsystem has changed 22 times since, so it is re-run under a witness: a signature now would certify superseded evidence. That is why it is worth separating from the rest rather than lumping everything into "not done".

Ranking is by a four-term priority score — must-ship weight, criticality, how badly the evidence fails, and how much has changed since — with each term visible in the extract so a reviewer can see which one is driving any given rank rather than trusting one opaque number.

09 · Data quality

The gate measures the share of **must-ship requirements whose readiness answer rests on data a check has flagged** — not a count of defective rows. A defect count tells you how untidy the data is; it does not tell you whether to trust the answer.

It passes at 1.779% against a 2.000% tolerance. That is a margin, not comfort, and the validation report says so rather than reporting a green light.

Eleven behavioural checks run; **three found nothing and are reported anyway**, because a summary listing only what was found cannot distinguish *we looked and it was clean* from *we never*

looked.

♦ **Policy breaches are deliberately excluded from the gate.** A safety requirement verified by its own owner is accurate data recording a practice that should not have happened. Its impact class is `Policy`; the KPI layer holds it to account. Conflating "the record is wrong" with "the practice is wrong" makes both harder to fix, and lets the gate be dismissed as noise.

Seven RAID items are due before they were raised — overdue on the day they were created. They are a data defect, they are reported as one, and `IsOverdue` explicitly excludes them. Counting them would inflate the overdue rate with a typing error and blame the programme for it.

10 · Management recommendations

#	Recommendation	Owner	Why
1	Do not ship build 88.	Programme board	135 of 281 must-ship requirements have no current, admissible evidence. This is not a judgement about risk appetite; it is that the evidence to make that judgement does not exist.
2	Freeze Flight Control and Release Mechanism, or accept that their verification cannot close.	Chief engineer	At 45 and 42 builds changed, re-running their suites buys evidence the next build invalidates. Freeze the interface, then verify.
3	Treat rig and airframe time as the schedule driver, not test throughput.	Programme board	425 rig hours outstanding against 180 a week, plus 272 airframe hours on one aircraft. A seventh rig shortens the rig work but not the flights; more test engineers shorten neither.
4	Countersign the 2 self-verified requirements a signature alone would fix; re-run the third under a witness.	Quality	REQ-0124 and REQ-0002 cost a second signature, not rig time: the cheapest readiness the programme can buy, and for these Safety items it is not optional. REQ-0488 passed on build 57 and its subsystem has changed 22 times since, so it needs a witnessed re-run.
5	Schedule the 110 under-levelled requirements explicitly.	Test manager	They are currently invisible: they read as passes. 62 need a rig slot and 12 a flight they have never been allocated, and 91 need a test written first at the level their type demands.

#	Recommendation	Owner	Why
6	Report readiness and completion side by side from now on.	Programme board	Not as a criticism of the existing metric. Reporting only completion is what let a gap of at least 42 points stay invisible for most of a year.
7	Re-baseline the seven RAID items due before they were raised.	PMO	Trivial to fix. They are excluded from the overdue count, but three are still open and counted as open, so they dilute the overdue rate the board uses (57.78%; 61.90% without them), and none of them can ever be reported overdue until re-baselined.

11 · What this analysis does not tell you

It does not forecast a date. 2.4 rig-weeks is a capacity statement, and the 272 airframe hours have no capacity to divide by. There is no critical path, no labour model and no cost, so "when will it be ready" is not answerable from this work.

Staleness is binary. A subsystem that changed once and one that changed eleven times both invalidate. The queue ranks by how much changed, but readiness itself does not award partial credit — deliberately, because a fractional ship gate is not a ship decision.

Test levels are assumed to be a ladder. Unit < Integration < HIL < Field assumes field evidence subsumes rig evidence. On a real programme some instrumented rig data cannot be replaced by a flight, and the policy table cannot currently express that.

One release candidate. The model is parameterised by build and the trend walks it backwards, so "would build 74 have been a better ship point" is answerable — it is simply not answered here.

All data synthetic. Talon Robotics is a fictional company. No confidential data or production-system claim is used anywhere in this analysis.